

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY
DSA521S - DATA STRUCTURES AND ALGORITHMS 1
PROJECT REPORT

NUST Service Centre Simulation:

Student Number	Full Name	Group Number
225047071	Tuwilika P Kanime	Grp1
225005255	Liswani LB	Grp2
226076350	Erastus P Nambala	Grp1
226034399	Pauline KZ Kufuna	Grp11
225006812	Christian Hatutale	
226141152	Werner AFT	Grp6

GitHub repository: <https://github.com/AndreasW-lab/DSA-MINI-PROJECT>

Table of Contents

1. Introduction
2. Part A - Data Structure Design
 - A1 Queue
 - A2 Singly Linked List
 - A3 Postfix Stack
 - A4 Array Statistics
 - A5 Data-Structure Justification
3. Part B - Sorting Algorithm Challenge
4. Part C - Algorithm Experiment
5. Part D - Integrated Service-Centre System
6. Part E - Pseudocode and Algorithm Representation
7. Conclusion
8. Submission Completion Checklist

1. Introduction

This project is a model of a campus service centre using implemented data structures and sorting algorithms using pseudocode and Java. The focus is correct selection, implementation, tracing and analysis of queues, singly linked lists, stacks, arrays, Selection Sort, Insertion Sort, Merge Sort and Quick Sort. The service-centre components are integrated into one menu-driven program, while postfix evaluation remains a separate stack exercise.

2. Part A - Data Structure Design

A1 - Waiting Line: Queue

A queue is appropriate because students should normally be served in the same order in which they arrive and a queue follows that exact principal, the First-In, First-Out principle.

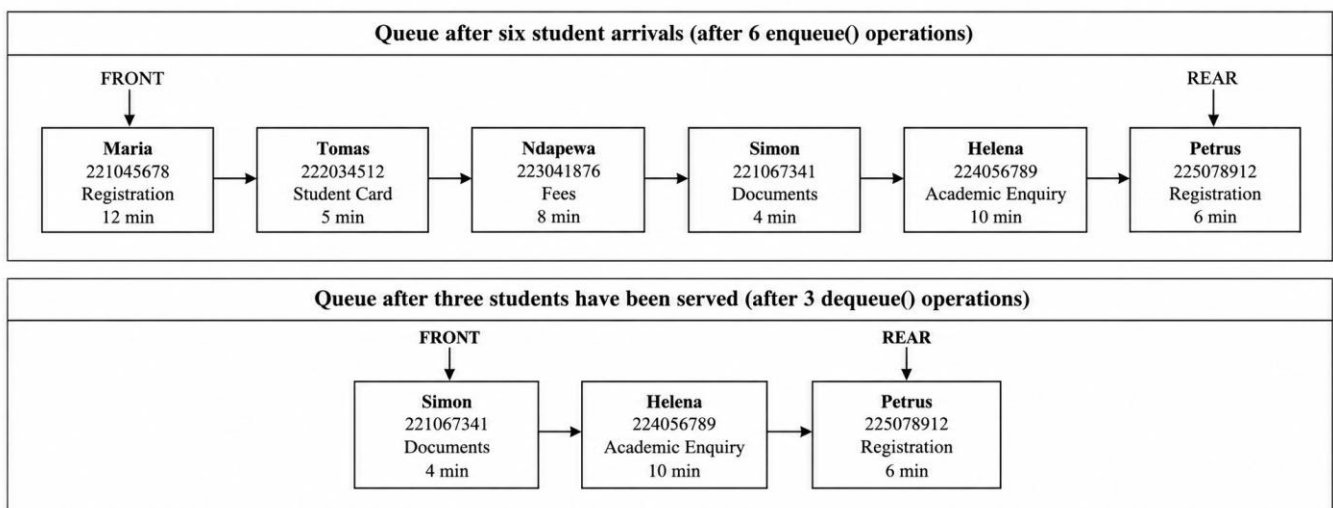


Figure A1. Queue after six arrivals and after three dequeue operations.

A1 pseudocode

```
ENQUEUE(student)
IF queue is full THEN
    DISPLAY "Queue is full"
    RETURN
END IF
rear <- next rear position
queue[rear] <- student
count <- count + 1
END

DEQUEUE()
IF isEmpty() THEN
    DISPLAY "Queue is empty"
    RETURN null
END IF
student <- queue[front]
front <- next front position
count <- count - 1
RETURN student
END

PEEK()
IF isEmpty() THEN RETURN null
RETURN queue[front]
END

ISEMPTY()
RETURN count = 0
END

DISPLAYQUEUE()
IF isEmpty() THEN
    DISPLAY "Queue is empty"
    RETURN
END IF
index <- front
FOR i <- 1 TO count
    DISPLAY queue[index]
    index <- next index position
END FOR
END
```

A2 - Student Service Records: Singly Linked List

Each node stores Student Number, Name, Service Type, Estimated Service Time, and a next reference. Required demonstrations include insertion at the beginning, end and a specified position, deletion, searching and traversal.

A2 - Before inserting Simon at the beginning

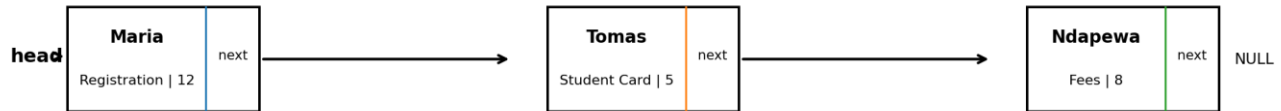


Figure A2.1. Before insertion at the beginning.

A2 - After inserting Simon at the beginning

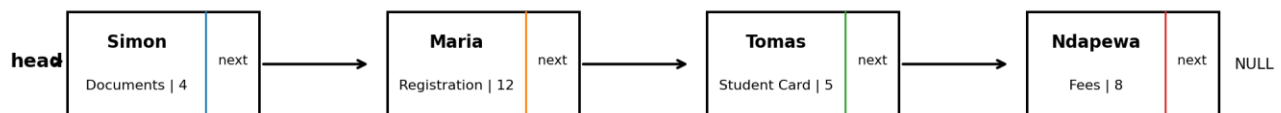


Figure A2.2. After inserting Simon at the beginning.

A2 - Before deleting Tomas

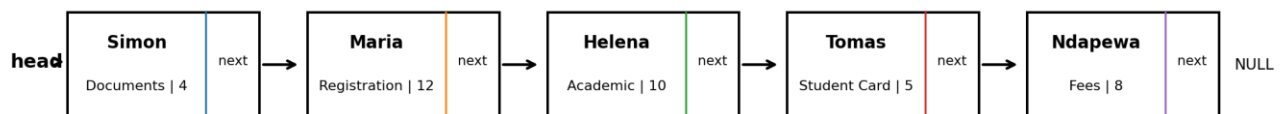


Figure A2.3. Before deleting Tomas.

A2 - After deleting Tomas

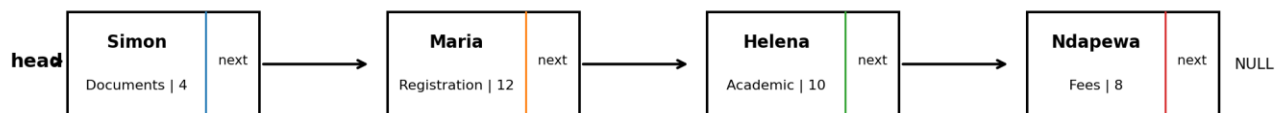


Figure A2.4. After deleting Tomas; links are reconnected.

A2 pseudocode

```
INSERT_BEGINNING(student)
newNode <- Node(student)
newNode.next <- head
head <- newNode
END

INSERT_END(student)
newNode <- Node(student)
IF head = null THEN head <- newNode; RETURN
current <- head
WHILE current.next != null
    current <- current.next
END WHILE
current.next <- newNode
END

INSERT_AT_POSITION(student, position)
IF position <= 1 THEN INSERT_BEGINNING(student); RETURN
newNode <- Node(student)
current <- head
FOR i <- 1 TO position - 2
    IF current = null THEN DISPLAY "Invalid position"; RETURN
    current <- current.next
END FOR
IF current = null THEN DISPLAY "Invalid position"; RETURN
newNode.next <- current.next
current.next <- newNode
END

DELETE(studentNo)
IF head = null THEN RETURN false
IF head.studentNo = studentNo THEN head <- head.next; RETURN true
current <- head
WHILE current.next != null AND current.next.studentNo != studentNo
    current <- current.next
END WHILE
IF current.next = null THEN RETURN false
current.next <- current.next.next
RETURN true
END

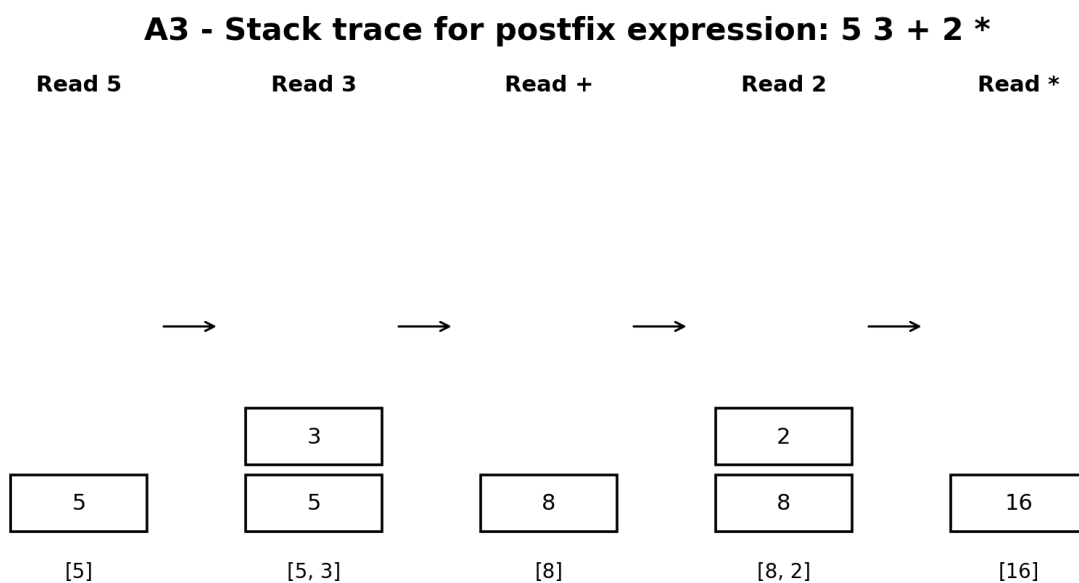
SEARCH(studentNo)
current <- head
WHILE current != null
    IF current.studentNo = studentNo THEN RETURN current
    current <- current.next
END WHILE
RETURN null
END

TRAVERSE()
current <- head
WHILE current != null
    DISPLAY current.student
    current <- current.next
END WHILE
END
```

A3 - Postfix Expression Evaluation: Stack

Algorithm Logic

Postfix notation (Reverse Polish Notation) places operators after their operands. To evaluate a postfix expression, we process the expression from left to right. When an operand (number) is encountered, it is pushed onto the stack. When an operator (+, -, x, ÷) is encountered, the top two operands are popped from the stack, the operation is performed, and the result is pushed back onto the stack. This process repeats until the entire expression has been read, leaving the final result as the only element remaining on the stack.



*Figure A3. Stack trace for 5 3 + 2 *.*

Final result: 16. The implementation must use push(), pop() and peek() and support +, -, * and /.

Stack Trace / Diagram (Required for A3)

For the expression $5\ 3 + 2 *$, the stack contents are shown after each major operation:

STEP	Token Read	Action Taken	Stack Contents(Bottom to Top)	Explanation
1	5	Push 5	[5]	Operand encountered, pushed to stack.
2	3	Push 3	[5, 3]	Operand encountered, pushed to stack.
3	+	Pop 3, Pop 5 $5 + 3 = 8$ Push 8	[8]	Operator encountered. Two operands popped, calculated, result pushed.
4	2	Push 2	[8, 2]	Operand encountered, pushed to stack.
5	*	Pop 2, Pop 8 $8 * 2 = 16$ Push 16	[16]	Operator encountered. Two operands popped, calculated, result pushed.
End	-	Pop 16	[]	End of expression. Final result is the last value remaining.

Execution Output

The following output was generated by the Java program when evaluating the expression $5\ 3 + 2 *$

```
PS C:\Dev\DSAProject> & 'C:\Program Files\Eclipse
Adoptium\jdk-25.0.4.101-hotspot\bin\java.exe' '-XX:
+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\User
s\User\AppData\Roaming\Code\User\workspaceStorage\b
8ed9fa2d9b3c7cedf30986fd0e26794\redhat.java\jdt_ws\
DSAProject_57eb5c39\bin' 'PostfixEvaluator'
Postfix Expression: 5 3 + 2 *
Final Result: 16
PS C:\Dev\DSAProject> |
```

A3 pseudocode

```
PUSH(value)
IF top = maxSize - 1 THEN
    DISPLAY "Stack overflow"
    RETURN
END IF
top <- top + 1
stack[top] <- value
END

POP()
IF top = -1 THEN
    DISPLAY "Stack underflow"
    RETURN null
END IF
value <- stack[top]
top <- top - 1
RETURN value
END

PEEK()
IF top = -1 THEN RETURN null
RETURN stack[top]
END

EVALUATE_POSTFIX(expression)
FOR each token IN expression
    IF token is a number THEN
        PUSH(token)
    ELSE
        operand2 <- POP()
        operand1 <- POP()
        result <- operand1 token operand2
        PUSH(result)
    END IF
END FOR
RETURN POP()
END
```

A4 - Daily Statistics: Array

The service-time array is traversed manually; no built-in sum, min or max shortcuts are used.

Statistic	Result
Service times	[12, 7, 15, 9, 22, 5, 11, 8, 14, 10, 6, 18]
Total students served	12
Total service time	137 min
Average service time	11.42 min
Highest	22 min
Lowest	5 min
Services > 10 min	6

A4 pseudocode

```
CALCULATE_STATISTICS(serviceTimes)
IF length(serviceTimes) = 0 THEN RETURN
total <- 0
highest <- serviceTimes[0]
lowest <- serviceTimes[0]
overTen <- 0
FOR each time IN serviceTimes
    total <- total + time
    IF time > highest THEN highest <- time
    IF time < lowest THEN lowest <- time
    IF time > 10 THEN overTen <- overTen + 1
END FOR
average <- total / length(serviceTimes)
DISPLAY total students, total, average, highest, lowest, overTen
END
```

A5 - Data-Structure Justification

Structure	Task-specific justification
Queue	FIFO preserves fair arrival order: new students join the rear and the longest-waiting student leaves from the front.
Singly Linked List	Dynamic records can be inserted or deleted by changing links rather than shifting a fixed array.
Stack	LIFO exposes the most recently read operands, matching postfix evaluation.
Array	Homogeneous service-time values can be traversed efficiently for daily statistics.

3. Part B - Sorting Algorithm Challenge

All four algorithms sort [17, 5, 23, 8, 14, 3, 11, 20, 6, 9] in ascending order without built-in sorting functions.

B1 - Selection Sort

Selection Sort repeatedly selects the smallest value from the unsorted portion and swaps it into the first unsorted position.

B1 - Selection Sort: first three passes										
Original	17	5	23	8	14	3	11	20	6	9
Pass 1	3	5	23	8	14	17	11	20	6	9
Pass 2	3	5	23	8	14	17	11	20	6	9
Pass 3	3	5	6	8	14	17	11	20	23	9

Figure B1. First three Selection Sort passes.

B1 pseudocode

```
SELECTION_SORT(array)
comparisons <- 0; swaps <- 0
FOR i <- 0 TO n-2
    minIndex <- i
    FOR j <- i+1 TO n-1
        comparisons <- comparisons + 1
        IF array[j] < array[minIndex] THEN minIndex <- j
    END FOR
    IF minIndex != i THEN SWAP array[i], array[minIndex]; swaps <- swaps + 1
    IF i < 3 THEN DISPLAY array
END FOR
END
```

B2 - Insertion Sort

Insertion Sort grows a sorted left portion. Larger elements are shifted right to make room for the key. The program must count both data-value comparisons and shifts.

B2 - Insertion Sort: first three passes										
Original	17	5	23	8	14	3	11	20	6	9
Pass 1	5	17	23	8	14	3	11	20	6	9
Pass 2	5	17	23	8	14	3	11	20	6	9
Pass 3	5	8	17	23	14	3	11	20	6	9

Figure B2. First three Insertion Sort passes.

B2 pseudocode

```
INSERTION_SORT(array)
comparisons <- 0; shifts <- 0
FOR i <- 1 TO n-1
    key <- array[i]; j <- i-1
    WHILE j >= 0
        comparisons <- comparisons + 1
        IF array[j] > key THEN array[j+1] <- array[j]; shifts <- shifts + 1; j <- j-1
        ELSE BREAK
    END WHILE
    array[j+1] <- key
    IF i <= 3 THEN DISPLAY array
END FOR
END
```

B3 - Merge Sort

Merge Sort recursively divides the array until each sub-array contains one element (the base case), then merges sorted sub-arrays.

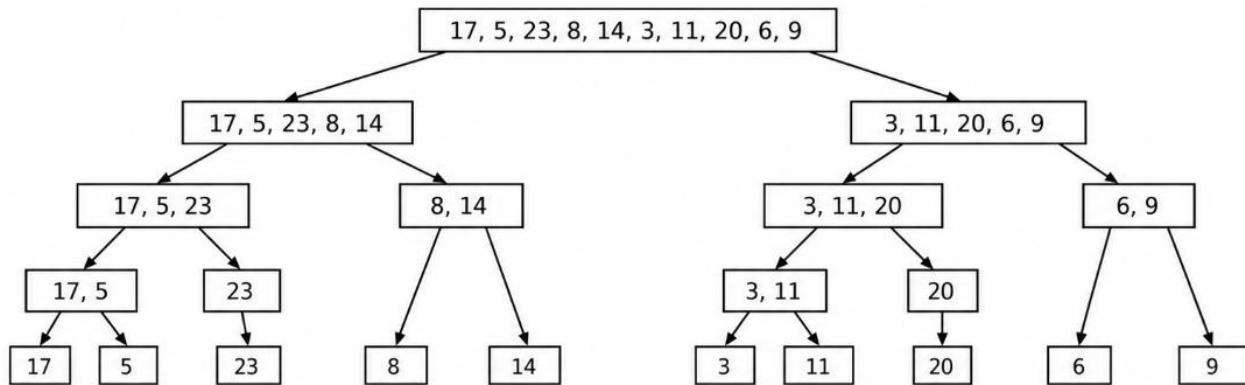


Figure B3. Complete divide-and-merge process with base case.

BASE CASE: Each sub-array contains one element and is already sorted.

$[17] + [5] \rightarrow [5, 17]$

$[5, 17] + [23] \rightarrow [5, 17, 23]$

$[8] + [14] \rightarrow [8, 14]$

$[3] + [11] \rightarrow [3, 11]$

$[3, 11] + [20] \rightarrow [3, 11, 20]$

$[6] + [9] \rightarrow [6, 9]$

$[5, 17, 23] + [8, 14] \rightarrow [5, 8, 14, 17, 23]$

$[3, 11, 20] + [6, 9] \rightarrow [3, 6, 9, 11, 20]$

FINAL:

$[5, 8, 14, 17, 23] + [3, 6, 9, 11, 20] \rightarrow [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]$

B3 pseudocode

```
MERGE_SORT(array, left, right)
IF left >= right THEN
    RETURN
END IF
mid <- (left + right) / 2
MERGE_SORT(array, left, mid)
MERGE_SORT(array, mid + 1, right)
MERGE(array, left, mid, right)
END

MERGE(array, left, mid, right)
CREATE leftArray from array[left ... mid]
CREATE rightArray from array[mid + 1 ... right]
i <- 0
j <- 0
k <- left
WHILE i < length(leftArray) AND j < length(rightArray)
    IF leftArray[i] <= rightArray[j] THEN
        array[k] <- leftArray[i]
        i <- i + 1
    ELSE
        array[k] <- rightArray[j]
        j <- j + 1
    END IF
    k <- k + 1
END WHILE
COPY remaining values from leftArray into array
COPY remaining values from rightArray into array
END
```

B4 - Quick Sort

The current submitted implementation uses the last element of the current partition as the pivot. The report and code must use the same pivot rule.

B4 - Quick Sort: first two partitioning stages

Pivot-selection rule: the last element of the current sub-array as the pivot.

Stage 1 Pivot = 9
Left partition: [5, 8, 3, 6]
Right partition: [23, 11, 20, 17, 14]

Stage 2 Recurse on left partition [5, 8, 3, 6]; pivot = 6
Left partition: [5, 3]
Right partition: [8]

Final sorted array: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]
--

Figure B4. First two Quick Sort partitioning stages.

B4 pseudocode

```
QUICK_SORT(array, low, high)
IF low < high THEN
    pivotIndex <- PARTITION(array, low, high)
    QUICK_SORT(array, low, pivotIndex - 1)
    QUICK_SORT(array, pivotIndex + 1, high)
END IF
END

PARTITION(array, low, high)
pivot <- array[high]
i <- low - 1
FOR j <- low TO high - 1
    IF array[j] <= pivot THEN
        i <- i + 1
        SWAP array[i], array[j]
    END IF
END FOR
SWAP array[i + 1], array[high]
RETURN i + 1
END
```

4. Part C - Algorithm Experiment

For each input size, one original random array is generated and copied so every algorithm receives the same values. Only data-value comparisons are counted and timing surrounds only the sorting call.

Algorithm	Input size	Number of comparisons	Execution time (ns)
Selection sort	20	190	7,075
Insertion sort	20	135	4,862
Merge sort	20	64	10,089
Quick sort	20	60	4,900
Selection sort	50	1,225	27,135
Insertion sort	50	599	15,352
Merge sort	50	217	19,054
Quick sort	50	229	10,546
Selection sort	100	4,950	97,840
Insertion sort	100	2,806	89,547
Merge sort	100	540	40,484
Quick sort	100	650	24,442
Selection sort	500	124,750	6,505,257
Insertion sort	500	62,525	3,852,012
Merge sort	500	3,860	403,268
Quick sort	500	5,139	193,604

Almost-sorted 100-element test

Additional Test Results: Almost sorted array Input size: 100		
Algorithm	Number of comparisons	Execution Time (ns)
Selection sort	4,950	11,485
Insertion sort	104	1,474
Merge sort	358	4,530
Quick sort	4,705	31,312

The almost-sorted array is created by sorting the 100-element data and swapping five neighbouring pairs, then copying the same resulting array to all four algorithms.

Part C experiment pseudocode

```
FOR each size IN [20, 50, 100, 500]
  original <- GENERATE_RANDOM_ARRAY(size)
  selectionInput <- COPY(original)
  insertionInput <- COPY(original)
  mergeInput <- COPY(original)
  quickInput <- COPY(original)
  FOR each algorithm
    RESET comparison counter
    start <- nanoTime()
    SORT copied input
    end <- nanoTime()
    RECORD size, algorithm, comparisons, end-start
  END FOR
END FOR

almostSorted <- sorted copy of the 100-element array
SWAP five neighbouring pairs
```

COPY the same almostSorted array for all four algorithms
RUN and RECORD the same measurements

Answers to Part C questions

1. Which algorithm performed the fewest comparisons?

Merge Sort performed the fewest comparisons overall for the larger input sizes. At an input size of 500, Merge Sort made 3,860 comparisons, compared with Quick Sort's 5,139, Insertion Sort's 62,525 and Selection Sort's 124,750.

2. Which algorithm performed the most comparisons?

Selection Sort performed the most comparisons on the random input. At an input size of 500, it made 124,750 comparisons, the highest recorded value in the main experiment.

3. Which algorithm performed best on the almost-sorted array?

Insertion Sort performed best on the almost-sorted array. It required only 104 comparisons and took 1,474 ns, which were the lowest values among the four algorithms for this test.

4. Which algorithm appeared to scale best as the input became larger?

Merge Sort and Quick Sort appeared to scale better as the input size increased. At 500 elements, Merge Sort required only 3,860 comparisons and Quick Sort required 5,139, while Selection Sort required 124,750 and Insertion Sort required 62,525.

5. Did the experimental results generally agree with the theoretical time complexity? Explain.

Yes. The results generally agreed with the expected time complexities. Selection Sort showed quadratic growth as the input size increased. Insertion Sort also required many more comparisons for larger random inputs, but performed very well on almost-sorted and already-sorted data, requiring only 104 and 99 comparisons, respectively. Merge Sort maintained relatively low comparison counts as the input increased. Quick Sort also performed well on random data but performed poorly on already-sorted data, where it required 4,950 comparisons, showing how its performance can be affected by the choice of pivot and the order of the input.

6. Is execution time alone enough to decide which algorithm is better? Explain.

No. Execution time alone is not enough because it can be affected by factors such as the computer's hardware, JVM performance and other processes running at the same time. The number of comparisons, input size, arrangement of the input data, time complexity and memory requirements should also be considered when comparing sorting algorithms.

5. Part D - Integrated Service-Centre System

Queue, Singly Linked List, Array and Sorting components operate inside one menu-driven Java program. The postfix stack remains separate. Serving a student removes the student from the queue, records the service time in the array and adds the student to the service-record linked list.

Part D integrated menu pseudocode

```
INITIALISE waitingQueue
INITIALISE serviceRecordList
INITIALISE serviceTimes

REPEAT
    DISPLAY "1. Add student to queue"
    DISPLAY "2. Serve next student"
    DISPLAY "3. Display waiting queue"
    DISPLAY "4. Add student record"
    DISPLAY "5. Display student records"
    DISPLAY "6. Search for student"
    DISPLAY "7. Delete student record"
    DISPLAY "8. Display daily statistics"
    DISPLAY "9. Sort service times"
    DISPLAY "10. Run sorting experiment"
    DISPLAY "11. Exit"
    READ choice
    CASE choice OF
        1:
            READ student details
            waitingQueue.ENQUEUE(student)
        2:
            student <- waitingQueue.DEQUEUE()
            IF student != null THEN
                ADD student.serviceTime TO serviceTimes
                serviceRecordList.INSERT_END(student)
                DISPLAY student
            ELSE
                DISPLAY "Queue is empty"
            END IF
        3:
            waitingQueue.DISPLAYQUEUE()
        4:
            READ student details
            serviceRecordList.INSERT_END(student)
        5:
```

```

        serviceRecordList.TRAVERSE()
6:
    READ studentNo
    result <- serviceRecordList.SEARCH(studentNo)
    IF result != null THEN
        DISPLAY result
    ELSE
        DISPLAY "Student not found"
    END IF
7:
    READ studentNo
    deleted <- serviceRecordList.DELETE(studentNo)
    IF deleted = true THEN
        DISPLAY "Student deleted"
    ELSE
        DISPLAY "Student not found"
    END IF
8:
    CALCULATE_STATISTICS(serviceTimes)
9:
    COPY serviceTimes TO sortArray
    DISPLAY "Choose sorting algorithm:"
    DISPLAY "1. Selection Sort"
    DISPLAY "2. Insertion Sort"
    DISPLAY "3. Merge Sort"
    DISPLAY "4. Quick Sort"
    READ sortChoice
    CASE sortChoice OF
        1: SELECTION_SORT(sortArray)
        2: INSERTION_SORT(sortArray)
        3: MERGE_SORT(sortArray, 0, length(sortArray) - 1)
        4: QUICK_SORT(sortArray, 0, length(sortArray) - 1)
        OTHERWISE:
            DISPLAY "Invalid sorting option"
    END CASE
    DISPLAY sortArray
    DISPLAY comparison count
10:
    RUN_SORTING_EXPERIMENT()
11:
    DISPLAY "Exiting program"
OTHERWISE:
    DISPLAY "Invalid option"

```

```
END CASE  
UNTIL choice = 11  
  
END
```


Stack - push/pop/postfix

```
PUSH(value)
IF top = maxSize - 1 THEN RETURN
top <- top + 1
stack[top] <- value
END

POP()
IF top = -1 THEN RETURN null
value <- stack[top]
top <- top - 1
RETURN value
END

EVALUATE_POSTFIX(expression)
FOR each token
  IF number THEN PUSH(number)
  ELSE operand2 <- POP(); operand1 <- POP(); PUSH(operand1 operator operand2)
END FOR
RETURN POP()
END
```

Singly Linked List

```
INSERT_NODE(student)
newNode <- Node(student)
IF head = null THEN head <- newNode; RETURN
current <- head
WHILE current.next != null: current <- current.next
current.next <- newNode
END

DELETE_NODE(studentNo)
IF head = null THEN RETURN false
IF head.studentNo = studentNo THEN head <- head.next; RETURN true
current <- head
WHILE current.next != null AND current.next.studentNo != studentNo
  current <- current.next
END WHILE
IF current.next != null THEN current.next <- current.next.next; RETURN true
RETURN false
END

SEARCH_NODE(studentNo)
current <- head
WHILE current != null
  IF current.studentNo = studentNo THEN RETURN current
  current <- current.next
END WHILE
RETURN null
END

TRAVERSE_LIST()
current <- head
WHILE current != null
  DISPLAY current.data
  current <- current.next
END WHILE
END
```

Selection Sort

```
FOR i <- 0 TO n-2
  minIndex <- i
  FOR j <- i+1 TO n-1
    comparisons <- comparisons + 1
    IF array[j] < array[minIndex] THEN minIndex <- j
  END FOR
  IF minIndex != i THEN SWAP; swaps <- swaps + 1
END FOR
```

Insertion Sort

```
FOR i <- 1 TO n-1
  key <- array[i]; j <- i-1
  WHILE j >= 0
    comparisons <- comparisons + 1
    IF array[j] > key THEN array[j+1] <- array[j]; shifts <- shifts + 1; j <- j-1
    ELSE BREAK
  END WHILE
  array[j+1] <- key
END FOR
```

Merge Sort

```
MERGE_SORT(array,left,right)
IF left >= right THEN RETURN // base case
mid <- (left + right) / 2
MERGE_SORT(array,left,mid)
MERGE_SORT(array,mid+1,right)
MERGE(array,left,mid,right)
END
```

Quick Sort

```
QUICK_SORT(array,low,high)
IF low < high THEN
  pivotIndex <- PARTITION(array,low,high)
  QUICK_SORT(array,low,pivotIndex-1)
  QUICK_SORT(array,pivotIndex+1,high)
END IF
END

PARTITION(array,low,high)
pivot <- array[high] // last-element pivot rule
i <- low - 1
FOR j <- low TO high-1
  comparisons <- comparisons + 1
  IF array[j] <= pivot THEN i <- i+1; SWAP array[i],array[j]
END FOR
SWAP array[i+1],array[high]
RETURN i+1
```